

Implementing Model Predictive Control on an Inverted Pendulum

Noah Parks

Advised by Professor Michael Piovoso

Swarthmore College

Engineering Design Capstone Project

Contents

Abstract	2
1 Introduction	3
2 Theory	5
2.1 Inverted Pendulum	5
2.2 Model Predictive Control	11
2.3 Discrete State Space Implementation	12
3 Methods	20
4 Results	21
4.1 State Space Representation	21
4.2 Simulink Block	24
4.3 Physical Model	26
5 Discussion	26
5.1 Further Work	27
5.2 Impacts	28
6 Conclusion	29
Acknowledgements	30
References	31
Appendix A: MATLAB code	33

Abstract

Inverted pendulums are a class of systems of great interest to engineers. Many systems requiring vertical stabilization can be modeled as one. They are a classic example of a nonlinear, unstable system and designing a controller to stabilize one is a common exercise in nonlinear control. Model predictive control, a control scheme introduced in the 1980s, is an optimal control strategy that has recently gained popularity and seen use in industries such as power electronics and agriculture. In this paper, we describe how we implement two such controllers on an inverted pendulum system. The first is a controller generated by MATLAB's Model Predictive Control Toolbox, and the second is an algorithm designed from scratch in MATLAB and Simulink. We test the controllers on two simulations of an inverted pendulum and a physical model to compare their performance.

1 Introduction

The inverted pendulum is a classic control problem often referenced in control theory textbooks and literature. In its simplest form, an inverted pendulum is a pendulum with its center of mass laying above its pivot point. A pencil balancing on one's finger is such a system. Despite how simple balancing an inverted pendulum may seem, it is by no means a trivial task. It provides an interesting control problem, and its popularity can be attributed to three main aspects: instability, non-linearity and applicability.

If a balanced inverted pendulum is disturbed, it is easy to imagine how it will continue to “fall over” rather than return to its balanced state. This is in obvious contrast to a classical pendulum; if a pendulum at rest is disturbed, it will oscillate around its rest position, eventually returning to its original position. We say the inverted pendulum is *unstable*: a bounded input causes the state of the system to diverge. Of course the state of such a system is limited by physical constraints, but the notion of instability remains.

It is a fairly straightforward exercise (one which will be omitted) to write down the governing equations of an inverted pendulum using the Euler-Lagrange equations. These equations, and the system, turn out to be non-linear. A nonlinear system is one that does not satisfy the *principle of superposition*:

Principle of Superposition: If $x_1(t)$ and $x_2(t)$ are arbitrary inputs to a

system S , and $y_1(t)$ and $y_2(t)$ their corresponding responses, then the response to the input $ax_1(t) + bx_2(t)$ where $a, b, \in \mathbb{R}$ is $ay_1(t) + by_2(t)$, if S is linear.

In other words, scaling the input to a linear system scales the response by the same amount, and summing two inputs results in a response that is the sum of the two individual responses. Similar to how linear differential equations are typically much easier to solve than nonlinear ones, linear systems are typically easier to model and control. However, most systems encountered in practice are nonlinear. As such, it is important that we are able to work with nonlinear systems to some extent.

Finally, the dynamics of an inverted pendulum can be applied to many systems of interest. The launching of a rocket can be thought of as a type of inverted pendulum. The Segway is a famous example of a commercial product that is nothing more than a rolling inverted pendulum. In fact, virtually every system supported from below requiring vertical stabilization can be analyzed as such a system. The human body and buildings are two such systems. Being able to understand, model and control an inverted pendulum means being able to do the same for these applications.

Model predictive control (MPC) has been used commercially as far back as the 1980s. It has found success in the chemical, oil and electronics industries due to its versatility and effectiveness. By predicting how the system will behave in the future, a model predictive controller is able to find the input that results in the “best” output. The “best” output is subjective and

depends on the needs of the controller designer. For example, it may be paramount that the system output converges to a certain value as quickly as possible, or maybe it's more important that the inputs remain small. This is all encoded in a loss function, and the optimal input is the one that minimizes this function. In this project, we implement two model predictive controllers on an inverted pendulum. The first is a controller generated using MATLAB's MPC Toolbox, and the second is a controller developed from scratch. Both are implemented using a combination of MATLAB and Simulink.

2 Theory

2.1 Inverted Pendulum

For our work we use the Quanser Rotary Inverted Pendulum, pictured below. The input is in the form of a voltage V_m applied to the servo motor. The motor rotates the horizontal rotary arm, whose position is defined as $\theta(t)$ and is measured from a given reference angle. The angle of the vertical pendulum is given by $\alpha(t)$ and is defined to be zero at the vertical.

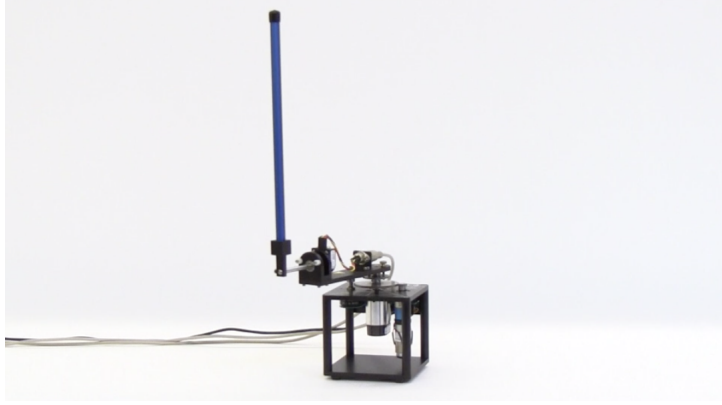


Figure 1: Quanser Rotary Inverted Pendulum

In order to develop a model predictive controller, we first need to derive a state space representation of the pendulum. This will be in the form

$$\dot{\mathbf{x}}(t) = A\mathbf{x}(t) + Bu(t)$$

$$\mathbf{y} = C\mathbf{x}$$

where $\mathbf{x} = [\theta \ \alpha \ \dot{\theta} \ \dot{\alpha}]^T$, A is a 4×4 constant matrix, B is a 4×1 vector, and

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

so that the output, $\mathbf{y}$ is $[\theta \ \alpha]^T$. The equations of motion of the pendulum are

given by

$$\begin{aligned} & \left(m_p L_r^2 + \frac{1}{4} m_p L_p^2 - \frac{1}{4} m_p L_p^2 \cos^2(\alpha) + J_r \right) \ddot{\theta} - \left(\frac{1}{2} m_p L_p L_r \cos(\alpha) \right) \ddot{\alpha} \\ & + \left(\frac{1}{2} m_p L_p^2 \sin(\alpha) \cos(\alpha) \right) \dot{\theta} \dot{\alpha} + \left(\frac{1}{2} m_p L_p L_r \sin(\alpha) \right) \dot{\alpha}^2 = \tau - D \dot{\theta} \end{aligned}$$

$$\begin{aligned} & - \frac{1}{2} m_p L_p L_r \cos(\alpha) \ddot{\theta} + \left(J_p + \frac{1}{4} m_p L_p^2 \right) \ddot{\alpha} - \frac{1}{4} m_p L_p^2 \cos(\alpha) \sin(\alpha) \dot{\theta}^2 \\ & - \frac{1}{2} m_p L_p g \sin(\alpha) = -D \dot{\alpha} \end{aligned}$$

where

$$\tau = \frac{K_g k_t (V_m - K_g k_m \dot{\theta})}{R_m}$$

The parameter names and values are listed below as found in [8]:

Parameter	Name	Value
m_p	Pendulum mass	0.127 kg
L_r	Rotary arm length	0.216 m
L_p	Pendulum length	0.337 m
J_r	Rotary arm moment of inertia	$9.98 \times 10^{-4} \text{ kg-m}^2$
J_p	Pendulum moment of inertia	$1.2 \times 10^{-2} \text{ kg-m}^2$
g	Gravitational acceleration	9.81 m-s^{-2}
K_g	Total gear ratio	70
k_t	Motor current-torque constant	$7.68 \times 10^{-3} \text{ N-m-A}^{-1}$
k_m	Motor back-emf constant	$7.68 \times 10^{-3} \text{ V-rad}^{-1}\text{-s}$
D	Viscous damping coefficient	$0.0024 \text{ N-s-m}^{-1}$
R_m	Motor armature resistance	$2.6 \text{ } \Omega$

Table 1: Parameters of Quanser Inverted Pendulum

The third equation represents the actuator dynamics of the system, i.e. it gives the conversion between the voltage input V_m and the torque generated by the motor τ .

Now, we need to linearize these equations. The linearization of a function $f(\mathbf{x})$ where $\mathbf{x} = [x_1 \ x_2 \ \dots \ x_n]^T$ about a point $\mathbf{x}_0 = [p_1 \ p_1 \ \dots \ p_n]^T$ is given

by

$$f_{\text{lin}} = f(\mathbf{x}_0) + \sum_{i=1}^n \frac{\partial f}{\partial x_i} \bigg|_{\mathbf{x}=\mathbf{x}_0} (x_i - p_i)$$

Applying this formula to our equations of motion using the point $[0 \ 0 \ 0 \ 0]^T$, we have

$$(m_p L_r^2 + J_r) \ddot{\theta} - \frac{1}{2} m_p L_p L_r \ddot{\alpha} = \tau - D \dot{\theta}$$

$$-\frac{1}{2} m_p L_p L_r \ddot{\theta} \left(J_p + \frac{1}{4} m_p L_p^2 \right) \ddot{\alpha} - \frac{1}{2} m_p L_p g \alpha = -D \dot{\alpha}$$

Now, we can put this in matrix-vector form, specifically $Cv = d$ where $v = [\ddot{\theta} \ \ddot{\alpha}]^T$ and solve for v by inverting A to arrive at the equations

$$\ddot{\theta} = \frac{1}{J_T} \left(- \left(J_p + \frac{1}{4} m_p L_p^2 \right) D \dot{\theta} - \frac{1}{2} m_p L_p L_r D \dot{\alpha} + \frac{1}{4} m_p^2 L_p^2 L_r g \alpha + \left(J_p + \frac{1}{4} m_p L_p^2 \right) \tau \right)$$

$$\ddot{\alpha} = \frac{1}{J_T} \left(\frac{1}{2} m_p L_p L_r D \dot{\theta} - (J_r + m_p L_r^2) D \dot{\alpha} + \frac{1}{2} m_p L_p g (J_r + m_p L_r^2) \alpha + \frac{1}{2} m_p L_p L_r \tau \right)$$

where $J_T = J_p m_p L_r^2 + J_r J_p + \frac{1}{4} J_r m_p L_p^2$. Now we can put the system in state

space representation:

$$\begin{aligned}
\begin{bmatrix} \dot{\theta} \\ \dot{\alpha} \\ \ddot{\theta} \\ \ddot{\alpha} \end{bmatrix} &= \frac{1}{J_T} \begin{bmatrix} 0 & 0 & J_T & 0 \\ 0 & 0 & 0 & J_T \\ 0 & \frac{1}{4}m_p L_p^2 L_r g & -(J_p + \frac{1}{4}m_p L_p^2) D & -\frac{1}{2}m_p L_p L_r D \\ 0 & \frac{1}{2}m_p L_p g (J_r + m_p L_r^2) & \frac{1}{2}m_p L_p L_r D & -(J_r + m_p L_r^2) D \end{bmatrix} \begin{bmatrix} \theta \\ \alpha \\ \dot{\theta} \\ \dot{\alpha} \end{bmatrix} \\
&+ \frac{1}{J_T} \begin{bmatrix} 0 \\ 0 \\ J_p + \frac{1}{4}m_p L_p^2 \\ \frac{1}{2}m_p L_p L_r \end{bmatrix} \tau
\end{aligned}$$

Substituting in the formula for τ and the values of the parameters, we arrive

at:

$$\begin{aligned}
\begin{bmatrix} \dot{\theta} \\ \dot{\alpha} \\ \ddot{\theta} \\ \ddot{\alpha} \end{bmatrix} &= \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 81.4033 & -10.2472 & -0.9319 \\ 0 & 122.0545 & -9.8605 & -1.3972 \end{bmatrix} \begin{bmatrix} \theta \\ \alpha \\ \dot{\theta} \\ \dot{\alpha} \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 83.4659 \\ 80.3162 \end{bmatrix} V_m \\
\begin{bmatrix} \theta \\ \alpha \end{bmatrix} &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} \theta \\ \alpha \\ \dot{\theta} \\ \dot{\alpha} \end{bmatrix}
\end{aligned}$$

the desired state-space representation.

2.2 Model Predictive Control

Model predictive control is an optimal control strategy, meaning the produced control signal is the solution to an optimization problem. Assume a plant's current state, $x(k)$, is known. Also assume that the controller is equipped with knowledge of the system to the degree that it can calculate the future state and output given the current state and an input signal. Typically, such controllers are discrete, so the input signal is constant over the $k+i$ 'th epoch $[(k+i)\Delta t, (k+i+1)\Delta t]$ where Δt is the sampling period. Let the prediction horizon, the number of epochs into the future we allow the controller to predict the system state and output, be n_p . Let U be an input signal over the prediction horizon, i.e. $U = [u(k) \ u(k+1) \ \dots \ u(k+n_p-1)]^T$ where $u(p)$ is the input signal over epoch p . We define a cost function $F(U, x(k))$ to be a measure of how "good" an input is. It typically takes the form

$$F(U, x(k)) = \sum_{i=1}^{n_p} y(k+i)^T Q y(k+i) + \sum_{j=0}^{n_p-1} u(k+j)^T R u(k+j)$$

where $y(k+i)$ is understood to mean the system output at epoch $k+i$ given the initial condition $x(k)$. Q and R are constant square matrices. Solving

the problem

$$\begin{aligned} & \arg \min_U F(U, x(k)) \quad \text{subject to} \\ & y_{min} \leq y(k+i) \leq y_{max}, i = 1, 2, \dots n_p \\ & u_{min} \leq u(k+j) \leq u_{max}, j = 0, 1, \dots n_p - 1 \end{aligned}$$

gives the optimal input signal U_{opt} . It may seem reasonable to pass this signal to the plant for all epochs in the prediction horizon. However, if an external force were to disturb the system's state, U_{opt} may no longer be the ideal signal. Instead, only the portion of U_{opt} over the next epoch is passed to the plant. After that epoch elapses, the plant's state is measured, and the process repeats with a new initial condition. Every iteration, the prediction horizon "slides" one epoch forward in time. Because of this, model predictive control is also known as sliding or receding horizon control. Re-evaluating the system and re-computing the optimal input signal every epoch is more computationally expensive than doing it once initially, but the controller becomes much more robust and resistant to external disturbances.

2.3 Discrete State Space Implementation

In our work, we have developed a model predictive controller from scratch. This implementation first requires the system as being described by a discrete state space representation:

$$\vec{x}[k+1] = A\vec{x}[k] + Bu$$

$$\vec{y}[k] = C\vec{x}[k]$$

where $\vec{x}[k]$ and $\vec{y}[k]$ represent the system state and output at epoch k , A, B and C are constant state matrices and u is the input, which is constant over a given epoch but allowed to vary between epochs. If there are n state variables and m output variables, then the dimensions of $\vec{x}[k]$, $\vec{y}[k]$, A , B and C are $n \times 1$, $m \times 1$, $n \times n$, $n \times 1$ and $m \times n$, respectively, while u is a scalar. We have also have $x[k] = x(k\Delta t)$ where Δt is the sampling time, a constant defined beforehand. To find a discrete state space representation for a system given the continuous state space representation, we solve for the system state $x(k)$ explicitly (note from this point on we will drop the vector arrows). We will do this using Laplace transforms. We proceed under the assumption that the Laplace transform can be applied to systems of differential equations in a way analogous to single differential equations. The following results will be important for our purposes:

$f(t) = \mathcal{L}^{-1}(F(s))$	$F(s) = \mathcal{L}(f(t))$
$x(t) \in \mathbb{R}^n$	$X(s) \in \mathbb{R}^n$
$x'(t)$	$sIX(s) - x(0), I \text{ } n \times n \text{ identity}$
$C \text{ constant}$	$C \frac{1}{s}$
$e^{At}, A \in \mathbb{R}^{n \times n}$	$(sI - A)^{-1}$
$\int_0^t f(\tau) d\tau$	$\frac{1}{s} F(s)$

Table 2: Laplace Transforms of Matrix/Vector Functions

For a more in-depth discussion, see [1]. Beginning with

$$x'(t) = Ax(t) + Bu$$

and taking the Laplace transform of both sides:

$$\begin{aligned}
sIX(s) - x(0) &= AX(s) + B \frac{u}{s} \\
(sI - A)X(s) &= x(0) + B \frac{u}{s} \\
X(s) &= (sI - A)^{-1}x(0) + (sI - A)^{-1}B \frac{u}{s}
\end{aligned}$$

Taking the inverse Laplace transform of both sides, we have

$$x(t) = e^{At}x(0) + \int_0^t e^{A\tau} d\tau Bu$$

If A is invertible, we can further simplify as

$$\begin{aligned}
x(t) &= e^{At}x(0) + \int_0^t I e^{A\tau} d\tau Bu \\
&= e^{At}x(0) + A^{-1} \int_0^t A e^{A\tau} d\tau Bu \\
&= e^{At}x(0) + A^{-1} \left(e^{A\tau} \Big|_0^t \right) Bu \\
&= e^{At}x(0) + A^{-1} (e^{At} - I) Bu
\end{aligned}$$

Note that we have used the assumption that u is constant from 0 to t , which is only true if $0 \leq t \leq \Delta t$. Specifically, we have

$$x(\Delta t) = A_d x(0) + B_d u$$

or

$$x[1] = A_d x[0] + B_d u \tag{3.1}$$

where $A_d = e^{A\Delta t}$ and $B_d = (A_d - I) A^{-1} B$. Note if A is singular, we may use its Moore-Penrose inverse. Therefore, we have found a state space representation for the system for the first sampling period, which we now wish to show generalizes. We introduce the notion of *time invariance*. Consider

a system S with two inputs, $x_1(t)$ and $x_2(t) = x_1(t - \tau)$, i.e. x_2 is x_1 delayed by some τ . Let the corresponding outputs be $y_1(t) = S\{x_1(t)\}$ and $y_2(t) = S\{x_2(t)\}$. Finally, assume the initial condition has also been time-delayed, i.e. $y_1(0) = y_2(\tau)$. We say S is time invariant if $y_2(t) = y_1(t - \tau)$ for all t . All systems represented by a linear ordinary differential equation with constant coefficients (or a system of such equations) are time invariant. So, we can directly apply this property to our example to conclude that

$$x(t + \Delta t) = A_d x(t) + B_d u$$

over any interval $[t, t + \Delta t]$ such that u is constant. In particular, this gives us

$$x((k + 1)\Delta t) = A_d x(k\Delta t) + B_d u$$

or

$$x[k + 1] = A_d x[k] + B_d u.$$

Because the output only depends on the input at the current time, the formula for the output remains the same, i.e.

$$y[k] = Cx[k]$$

giving us our desired representation. Now, we can explicitly find the output of the system at every epoch in terms of the state matrices as follows. First, define n_p to be the prediction horizon, the number of epochs in the future we want to predict the system output for. Also let n_c to be the control horizon, the number of epochs into the future we wish to optimize the control sequence over. We must have $n_p \geq n_c$, and we will let them be equal in our case, so let $n = n_p = n_c$. Assume the initial state of the system, $x[k]$ is known, and let $u[k]$ be the control input over the k 'th epoch. Then,

$$\begin{aligned}
x[k+1] &= A_d x[k] + B_d u[k] \\
x[k+2] &= A_d x[k+1] + B_d u[k+1] \\
&= A_d^2 x[k] + A_d B_d u[k] + B_d u[k+1] \\
&\vdots \\
x[k+n] &= A_d^n x[k] + A_d^{n-1} B_d u[k] + A_d^{n-2} B_d u[k+1] + \dots + B_d u[k+n-1]
\end{aligned}$$

Since $y = Cx$ for all epochs, we have

$$\begin{aligned}
y[k+1] &= Cx[k+1] \\
&= CA_d x[k] + CB_d u[k] \\
y[k+2] &= CA_d^2 x[k] + CA_d B_d u[k] + CB_d u[k+1] & \vdots \\
y[k+n] &= CA_d^n x[k] + CA_d^{n-1} B_d u[k] + CA_d^{n-2} B_d u[k+1] + \dots \\
&\quad + CB_d u[k+n-1]
\end{aligned}$$

Now, if we define

$$Y = \begin{bmatrix} y[k+1] & y[k+2] & \dots & y[k+n] \end{bmatrix}^T,$$

$$U = \begin{bmatrix} u[k] & y[k+1] & \dots & y[k+n-1] \end{bmatrix}^T$$

then we have $Y = Fx[k] + \Phi U$ where

$$F = \begin{bmatrix} CA_d \\ CA_d^2 \\ \vdots \\ CA_d^n \end{bmatrix}, \Phi = \begin{bmatrix} CB_d & 0 & 0 & \dots & 0 \\ CA_d B_d & CB_d & 0 & \dots & 0 \\ CA_d^2 B_d & CA_d B_d & CB_d & \dots & 0 \\ \vdots & & & & \\ CA_d^{n-1} B_d & CA_d^{n-2} B_d & CA_d^{n-3} B_d & \dots & CB_d \end{bmatrix}$$

Now that we have an explicit formula for the system output, we can optimize the control input. Let r be the target vector of the system output, and let $R = [r \ r \ \dots \ r]^T$ where there are n elements. Now we define the cost function to be minimized:

$$F = ||R - Y||_2^2 + w||U||_2^2$$

The first term is a measure of how close the output gets to the target output. The second term measures how large the control input gets. w is a non-negative weight that determines how much emphasis will be put on the second term during the optimization. If w is small, the optimal U will be one that

gets Y very close to R , regardless of how large it becomes. As w increases, the optimal U will become smaller at the cost of not converging to the target output as quickly.

To minimize F , we express it as

$$\begin{aligned}
F &= ||R - Y||_2^2 + w||U||_2^2 \\
&= (R - Y)^T(R - Y) + U^T W U \\
&= (R - (Fx[k] + \Phi U))^T(R - (Fx[k] + \Phi U)) + U^T W U \\
&= (R - Fx[k])^T(R - Fx[k]) - 2U^T \Phi^T(R - Fx[k]) + U^T \Phi^T \Phi U + U^T W U \\
&= (R - Fx[k])^T(R - Fx[k]) - 2U^T \Phi^T(R - Fx[k]) + U^T(\Phi^T \Phi + W)U
\end{aligned}$$

where $W = \text{diag}(w)$. Taking the derivative with respect to U and setting equal to 0:

$$\begin{aligned}
\frac{\partial F}{\partial U} &= 0 = -2\Phi^T(R - Fx[k]) + 2(\Phi^T \Phi + W)U \\
\implies U &= (\Phi^T \Phi + W)^{-1} \Phi^T(R - Fx[k])
\end{aligned}$$

This vector minimizes our cost function. We take the first element as the input to the system, update the system state at the next epoch, and repeat until the system is controlled to its desired output. In our implementation, we are constrained by the maximum voltage the motor can receive. We do not expect this to be an issue, and having the input saturate at this value is a simple fix should it occur.

3 Methods

To test the efficacy of the two controllers, we implement them on three representations of the inverted pendulum: the state-space representation, the Simulink block, and the physical system. The state space representation derived in section 2.1 gives a reasonable approximation of the true system near the operating point the system was linearized about, $[0 \ 0 \ 0 \ 0]$. As long as the pendulum remains relatively vertical and the angle doesn't change too much, it should be a decent indication of how the controllers will perform on the true model. We are able to plot the input and output of the system to quantitatively compare the effectiveness of the two controllers.

QUARC, a software package for MATLAB/Simulink developed by Quanser, includes a Simulink block that models the inverted pendulum. If an input is passed to this block, it outputs the four state variables. Note that the block takes in a torque as an input, so we need to manually implement the pendulum's actuator dynamics in order to convert a voltage to a torque. This representation of the system has the advantage of modelling the system's non-linear behaviour, unlike the state space representation. Similar to the state space representation, we plot the input and output to compare the controllers.

Finally, we have the physical inverted pendulum system. Obviously the physical system exhibits the same non-linearities as the Simulink block. Unlike the Simulink models, computation time is now a concern. If the

controller does not run fast enough, the controller would lag behind the pendulum, potentially leading to delayed or incorrect inputs which may result in the pendulum falling over. This was not a concern with the other two models due to how the simulation waits for the controller to pass an input before updating. Our analysis of the performance of the controllers on the physical pendulum will be more qualitative than the previous models. We vary the initial pendulum angle to find the maximum angle such that the controller can consistently control the pendulum to vertical.

4 Results

Both controllers were created using the continuous state space representation of the inverted pendulum derived in section 2.1. For the discrete state space implementation, we let w , the weight that determines how much we value the input signal's magnitude being minimized, be 1, the sampling time be 0.1 seconds and the prediction horizon be 10 epochs. For all trials using the first two system representations we use the initial condition $[0 \ 0.3 \ 0 \ 0]^T$.

4.1 State Space Representation

The Simulink model used to test the MATLAB-generated controller on the state-space model of the pendulum is shown below.

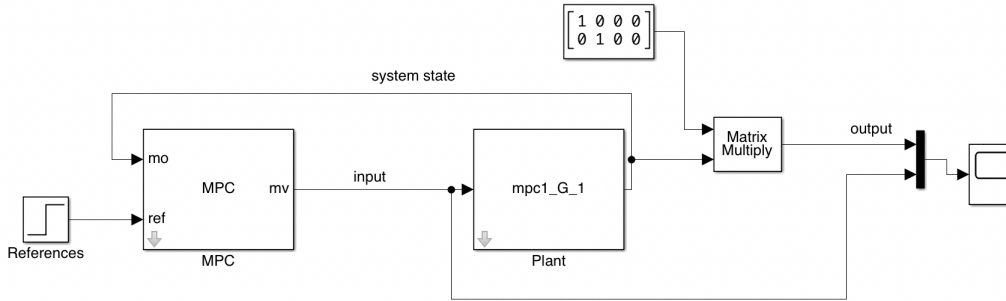


Figure 2: Simulink Model of MATLAB Generated Controller

Using this setup, we have the following plot of the pendulum angle and system input:

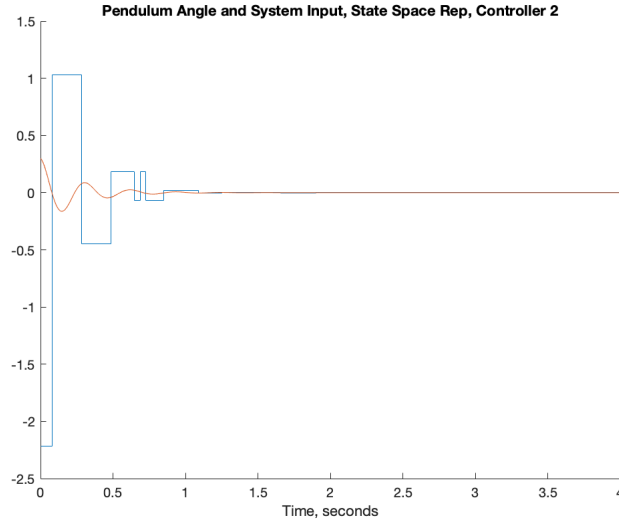


Figure 3: Pendulum angle (orange, rad) and system input (blue, V) for the MATLAB controller on the state space representation

From the figure, we measure a 57.2 percent overshoot (how much the pendulum angle overshoots the target value as a percentage of the distance from the initial condition and the target value) and a 5% settling time (the time at which the pendulum angle is within 5% of the true value for all time afterwards) of 1.13 seconds.

The model of the discrete space controller is in the following figure.

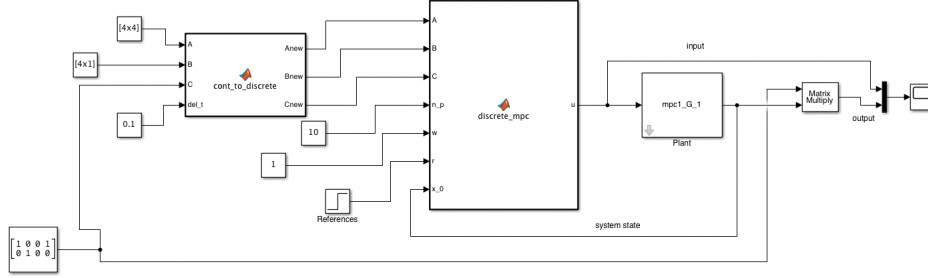


Figure 4: Simulink Model of Discrete State Space Controller

The code for the two MATLAB functions, which converts a continuous state space model into a discrete time one, and find the optimal control input as described in section 2.3 respectively, can be found in Appendix A. The model gives us the following plot:

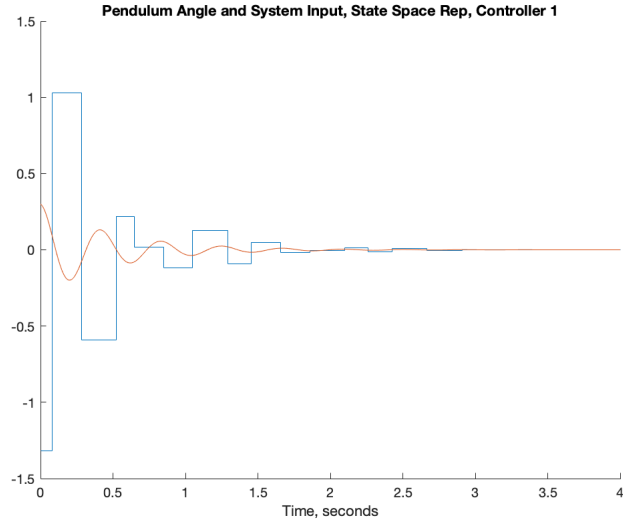


Figure 5: Pendulum angle (orange, rad) and system input (blue, V) for the discrete controller on the state space representation

We observe a 63.2 percent overshoot and a 5% settling time of about 1.78 seconds.

4.2 Simulink Block

To test the controllers on the Simulink block model of the inverted pendulum, we use the same Simulink diagrams as the previous section, replacing the state space model with the block. Because the block takes in a torque input, we need to add actuator dynamics to convert a voltage input to torque. The results given by the MATLAB controller and discrete controller, respectively follow:

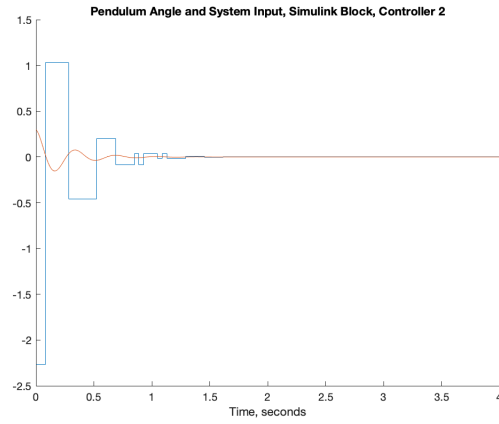


Figure 6: Pendulum angle (orange, rad) and system input (blue, V) for the MATLAB controller on the Simulink block model

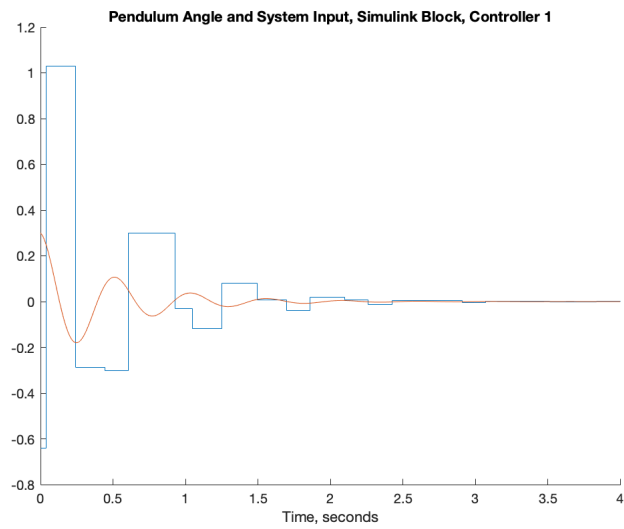


Figure 7: Pendulum angle (orange, rad) and system input (blue, V) for the discrete controller on the Simulink block model

The MATLAB controller had a percent overshoot and 5% settling time of 61.4 and 1.15 seconds, respectively, while the discrete controller gives 65.1% and 2.03 seconds.

4.3 Physical Model

Both controllers successfully controlled the physical pendulum to vertical given a nonzero initial condition. The MATLAB controller was able to do so consistently if the pendulum had an initial angle of about ± 0.85 rad or less. The discrete controller succeeded in stabilizing the pendulum if its initial angle was about ± 0.75 rad or less.

5 Discussion

Both controllers were able to control all three representations of the inverted pendulum. The MATLAB-generated controller consistently outperformed the discrete controller with smaller overshoots and settling times, as well as a wider range of initial conditions from which it could control the physical pendulum. To improve the performance of our controller, we could empirically optimize the values of n_p and w , although the performance would not be expected to improve drastically. Increasing n_p would also increase the computation time of the controller, possibly causing it to lag behind the system resulting in the system output diverging, i.e. the pendulum falling over. Regardless, our controller was successful in its task.

5.1 Further Work

A natural extension of our work is the implementation of *swing-up control*. Throughout this project, the inverted pendulum systems have small initial conditions, i.e. they start from near vertical. However, the inverted pendulum is at equilibrium when it is hanging down like a classic pendulum, which seems like a more natural initial condition. Controlling an inverted pendulum from such an initial condition is known as swing-up control. Before the stabilization control we have implemented can be applied to the pendulum, it first must be “swung up”. Doing so requires the rotary arm to swing back and forth, increasing the pendulum’s kinetic energy until it is able to reach an upright position. The movement of the rotary arm must be carefully controlled to induce resonance in the pendulum, otherwise the pendulum will not have enough kinetic energy to overcome gravity and friction and reach an upright position. This can be readily accomplished by energy-based control strategies, i.e. systems that control the pendulum’s energy rather than its position and velocity directly. Model predictive control has also been successfully applied to this problem (see [9]) and such a project would be a reasonable next step for our work.

In our development of a from-scratch controller, we derived the discrete state-space representation of the system and used it to define a cost function to be minimized. The result is a discrete vector of inputs to be applied over a given epoch. Instead, we could design an analogous continuous state-space

controller. Rather than explicitly solving for the system output at given times, we would allow a continuous input signal and therefore a continuously varying output. The cost function would be similar to the function we defined for the discrete controller, with the summations replaced with integrals over the prediction horizon. Such a controller would be more robust since the input would vary continuously rather than changing at given moments at the cost of computational efficiency. Our controller need only solve a quadratic program, i.e. the cost function is quadratic in the variable we are optimizing over. Minimizing an integral is a much harder and computationally intensive task.

5.2 Impacts

It is not expected that our work will immediately lead to societal, environmental or economic impacts. The software and hardware used for this project is commercially available, and is produced for the sole purpose of education. We do understand that significant physical resources that went into producing the hardware. However, model predictive control is, and will continue to be, applied in numerous ways that affect society or the environment. Since the 1980s, model predictive control has been used to regulate oil production and other chemical processes that may directly harm the environment. However, it has also seen use in applications beneficial to the environment and society. Recently, model predictive control has been implemented in agricultural applications, especially irrigation to regulate the amount of wa-

ter dispensed to crops (see [6]). This is done to ensure that crops receive the necessary amount of water without being over-watered, cutting down on waste.

6 Conclusion

We were successful in implementing model predictive control on an inverted pendulum system. Both controllers were able to control all three models, with the MATLAB-generated controller outperforming ours. This is to be expected, as MATLAB's Model Predictive Control Toolbox is significantly more optimized than our implementation.

Further work will be required to improve the performance of our algorithm. One such advancement may be utilizing MATLAB's compatibility with C compilers. Running our code in C would speed up computation times and allow for more intensive optimization calculations without fear of the controller lagging behind the system. It is expected that further optimization of our implementation would lead to performance comparable to the MATLAB-generated controller.

Inverted pendulum systems are found in a surprising number of engineering applications, so it is important that we understand their dynamics and how to effectively control them. In our work, we have shown that a relatively simple algorithm is able to control such a system.

Acknowledgements

I would like to thank Professor Michael Piovoso for his expertise and support throughout this project, as well as Ed Jaoudi for his technical support and help with software licensing.

References

- [1] Louis A.Pipes. “Applications of Laplace transforms of matrix functions”. In: *Journal of the Franklin Institute* 285.6 (1968 [Online]. doi: [https://doi.org/10.1016/0016-0032\(68\)90049-5](https://doi.org/10.1016/0016-0032(68)90049-5)).
- [2] Masood Askari et al. “Model predictive control of an inverted pendulum.” In: Jan. 2010, pp. 1–4. DOI: 10.1109/TECHPOS.2009.5412069.
- [3] Alberto Bemporad, N. Lawrence Ricker, and Manfred Morari. *Model Predictive Control Toolbox: Getting Started Guide*. Natick, MA, USA, 2021 [Online].
- [4] Alberto Bemporad, N. Lawrence Ricker, and Manfred Morari. *Model Predictive Control Toolbox: User Guide*. Natick, MA, USA, 2021 [Online].
- [5] Petr Chalupa and Vladimir Bobal. “Modelling And Predictive Control Of Inverted Pendulum”. In: (June 2008). DOI: 10.7148/2008-0531.
- [6] Ying Ding et al. “Model predictive control and its application in agriculture: A review”. In: *Computers and Electronics in Agriculture* 151 (Aug. 2018), pp. 104–117. DOI: 10.1016/j.compag.2018.06.004.
- [7] Quanser Inc. *Rotary Pendulum Workbook, Instructor Version*. Markham, Ontario, Canada, 2011 [Online].
- [8] Quanser Inc. *User Manual: SRV02 Rotary Servo Base Unit*. Markham, Ontario, Canada, 2011 [Online].
- [9] Sooyong Jung and J.T. Wen. “Nonlinear Model Predictive Control for the Swing-Up of a Rotary Inverted Pendulum”. In: *Journal of Dynamic*

- Systems, Measurement, and Control* 126 (Sept. 2004). DOI: 10.1115/1.1789541.
- [10] J.B. Rawlings, D.Q. Mayne, and M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2017. ISBN: 9780975937730. URL: <https://books.google.com/books?id=MrJctAEACAAJ>.
- [11] Quan Truong. “Continuous-time Model Predictive Control”. M.S. thesis. Melbourne, Australia: RMIT University, 2007 [Online]. URL: <https://core.ac.uk/download/pdf/15614981.pdf>.

Appendix A: MATLAB Code

```
function [Anew,Bnew,Cnew] =  
cont_to_discrete(A,B,C,del_t)  
  
%calculate A_d  
Anew = expm(A*del_t);  
  
%calculate B_d using inverse of A or Moore-Penrose  
%inverse depending on if A is singular  
if rank(A) < size(A,1)  
    Bnew = (Anew * eye(size(Anew,1)))*pinv(A)*B;  
else  
    Bnew = (Anew * eye(size(Anew,1)))*inv(A)*B;  
end  
Cnew = C;  
  
function u = discrete_mpc(A, B, C, n_p, w, r, x_0)  
  
%calculate theta matrix  
size1 = size(C*B,1);  
size2 = size(C*B,2);  
theta = zeros([n_p*size1,n_p*size2]);  
for i = 1:n_p  
    for j = 1:n_p  
        if i-j<0  
            theta(size1*(i-1)+1:size1*i,size2*(j-1)
```

```

+1:size2*j)=0;

else
    theta(size1*(i-1)+1:size1*i,size2*(j-1)
+1:size2*j) = C*A^(i-j)*B;
end
end
end

%calculate F matrix
size3 = size(C*A,1);
size4 = size(C*A,2);

F = zeros([n_p*size3,size4]);
for i = 1:10
    F(size3*(i-1)+1:size3*i,:) = C*A^(i);
end

%calculate W (weight) matrix
W = diag(w*zeros(1,np));

%define R (reference signal vector)
R = repmat(r,size(theta3,1),1);

```

```

%find optimal input signal
U = (theta.'*theta-W)\theta.'*(R-F*x_0);

%output first epoch of optimal input, saturate at
    +/-5 if necessary
if U(1) > 5
    u = 5;
elseif U(1) < -5
    u = -5;
else
    u = U(1);
end

```